

Sauf mention contraire, les fenêtres graphiques utilisées seront de taille 900×600.

1 Prise en main de `graphics_nsi`

Exercice 1.1 (Exemple introductif) Tapez le code suivant :

```
# -*- coding: utf-8 -*-
# Exercice 1.1 : exemple introductif

# Importation du module graphique graphics_nsi
from graphics_nsi import *

# Fonction principale
def main():
    # Initialisation de la fenêtre graphique
    init_graphic(900,600,"Exemple introductif")

    # Taper votre code ci-dessous
    P=Point(100,90)
    draw_fill_rectangle(P,50,25,blue)

    P1=Point(10,10)
    P2=Point(400,500)
    draw_line(P1,P2,red)

    draw_circle(Point(300,350),100,magenta)
    # Fin de votre code

    # Permet d'attendre un appui sur Echap avant de fermer la fenêtre graphique
    wait_escape()
    return 0

# Exécute la fonction principale après interprétation
main()
```

Exécutez ce code puis établissez le lien entre celui-ci et l’affichage obtenu.

Exercice 1.2 (Quelques tracés...) Tracez les dessins suivants :

1. Un segment horizontal et trois cercles : le premier centré sur l’extrémité gauche du segment, le deuxième centré au milieu du segment et le troisième centré sur l’extrémité droite du segment.
2. Cinq lignes horizontales, les unes au dessus des autres et espacées de 100.
3. Un carré de côté 100 dont le coin en haut à gauche est aux coordonnées (20 ; 20).
4. Une croix (style ×) formée de deux segments et insérée dans un carré de côté 50 dont le centre est en (120 ; 170).

Exercice 1.3 (Croix)

1. Ecrivez une fonction `dessine_croix(centre, largeur, couleur)` qui dessine la croix centrée sur le point `centre` et dont le carré englobant a un côté de taille `largeur`.
2. Testez cette fonction par différents appels dans la fonction `main`.

Exercice 1.4 (Quadrillage) Programmez chacune des trois fonctions suivantes et testez les par différents appels dans la fonction `main` :

1. la fonction `DesLignesH(C, d)` qui dessine des lignes horizontales de couleur `C` espacées de `d` sur toutes la hauteur de la fenêtre graphique,
2. la fonction `DesLignesV(C, d)` qui dessine des lignes verticales de couleur `C` espacées de `d` sur toute la largeur de la fenêtre graphique,
3. la fonction `Quadrillage(C, d)`, utilisant les deux fonctions précédentes, qui dessine un quadrillage de couleur `C` dont les lignes horizontales sont espacées de `d` et les lignes verticales également.

Exercice 1.5 (Echiquier)

1. Représenter un échiquier (cases noires et blanches) de 8 cases par 8 cases (prendre une fenêtre graphique de taille 800×800).
2. Le code ne doit comporter *aucun* appel à la fonction `draw_line()` et *qu'un seul* appel à la fonction `draw_fill_rectangle()`.

Exercice 1.6 (Animation 1) Affichez un cercle bleu de rayon 50, puis simulez son déplacement sur le bord inférieur de la fenêtre graphique en l'effaçant et en le réaffichant successivement.

2 Gestion de la souris

Exercice 2.1 (Triangle)

1. Ecrivez une fonction `dessine_triangle(P1, P2, P3)` qui dessine le triangle formé par les trois points passés en arguments, le segment reliant P1 et P2 étant bleu, celui entre P2 et P3 étant jaune et celui entre P3 et P1 étant blanc.
2. Testez cette fonction par différents appels dans la fonction `main` qui contiendra le code principal suivant :

```
P1=wait_click()
P2=wait_click()
P3=wait_click()
dessine_triangle(P1,P2,P3)
```

3. Que se passe-t-il si on change dans le `main` l'appel de la fonction par `dessine_triangle(P2,P3,P1)` ?

Exercice 2.2 (Gestion de la souris) Ecrivez une fonction permettant d'effectuer les actions suivantes :

1. Attendre un clic (utilisez la commande `wait_click()`) et dessiner un cercle de rayon 100 là où l'utilisateur a cliqué.
2. Répéter cette enchaînement cinq fois.

Exercice 2.3 (A droite ou à gauche ?) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher une ligne verticale en blanc qui sépare l'écran en deux parties égales.
2. Attendre un clic de l'utilisateur. Si le clic est à gauche de la ligne, afficher un cercle bleu et s'il est à droite, afficher un cercle rouge. Dans les deux cas, le cercle doit être centré là où l'utilisateur a cliqué.
3. Modifier l'étape 2 de sorte que cette fois, si le point cliqué P est à gauche, on affichera un cercle bleu de centre le point symétrique de P par rapport à la ligne verticale. De même si le point cliqué P est à droite, mais le cercle doit être jaune.

Exercice 2.4 (Deux cercles ou un segment ?) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher une ligne verticale en jaune qui sépare l'écran en deux parties égales.
2. Attendre deux clics. Si les deux clics sont chacun d'un côté de la ligne verticale, afficher deux cercles bleus centrés en ces points. Si les deux clics sont du même côté de la ligne verticale, afficher un segment rouge reliant ces deux points.

Exercice 2.5 (Un triangle sinon rien) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher une ligne verticale en blanc qui sépare l'écran en deux parties égales.
2. Attendre trois clics. Si les trois clics sont du même côté de la ligne, afficher un triangle reliant les trois points. Sinon, ne rien afficher. *On pourra utiliser la fonction `dessine_triangle` définie dans l'exercice 2.1.*

Exercice 2.6 (Triple zone) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher deux lignes verticales séparant l'écran en trois zones de même taille. *On pourra utiliser la fonction `DesLignesV` définie dans l'exercice 1.4.*
2. Attendre un clic. Si le clic est dans la zone de gauche, afficher un cercle bleu, s'il est dans la zone du milieu, afficher un cercle blanc et s'il est dans la zone de droite, afficher un cercle rouge.
3. Répéter l'enchaînement de la question 2 trois fois.

Exercice 2.7 (Cercles et parité) Ecrivez une fonction réalisant les constructions suivantes :

1. Attendre un clic de l'utilisateur puis afficher un cercle centré au point P cliqué et dont la couleur dépend de la parité des coordonnées de P :

	abscisse paire	abscisse impaire
ordonnée paire	rouge	bleu
ordonnée impaire	jaune	vert

On utilisera le modulo % pour tester la parité.

2. Répéter l'enchaînement précédent quatre fois.

Exercice 2.8 (Disques et couleurs 1) Ecrivez une fonction `dessine_disque_couleur(rayon)` permettant de réaliser la séquence suivante :

1. Attendre un clic.
2. Dessiner un disque de couleur rouge, centré au point cliqué et de rayon `rayon`.
3. Attendre un clic.
4. Si le clic est à l'intérieur du disque, on change la couleur en bleu et on recommence... pour afficher la séquence rouge, bleu, jaune, rouge,...
5. Si le clic est à l'extérieur du disque, la fonction se termine.

Exercice 2.9 (Disques et couleurs 2) Ecrivez une fonction réalisant les constructions suivantes :

1. Séparer l'écran en quatre parties égales.
2. Attendre un clic.
3. Afficher un disque bleu (resp. rouge, vert, jaune) si le point cliqué est en bas à gauche (resp. en bas à droite, en haut à gauche, en haut à droite).
4. Répéter cet enchaînement cinq fois.

On écrira en particulier les fonctions :

- * `trouve_lieu(P)` qui renvoie la valeur 0, 1, 2 ou 3 suivant que le point P est dans l'un des quatre quadrants.
- * `disque_couleur(centre)` qui affiche le disque voulu.

Exercice 2.10 (Disques et couleurs 3)

1. Écrivez les fonctions suivantes :
 - * `dessine_couleurs()` qui dessine quatre carrés de couleurs différentes en bas à droite de l'écran.
 - * `choix_couleur(P)` qui renvoie, si le point P est dans un des carrés, la couleur de ce carré et qui renvoie `noir` sinon.
 - * `dessine_disque()` qui attend des clics ; si le clic est dans un des quatre carrés, elle stocke la couleur de ce carré dans une variable courante `C` ; si le clic est en dehors des quatre carrés, elle dessine un disque de la couleur courante.
2. Ecrivez la fonction `main` de telle sorte que le programme s'arrête au bout de 20 disques dessinés.

Exercice 2.11 (Animation 2) Modifiez la fonction de l'exercice 7 afin que le cercle se déplace le long des bords de la fenêtre graphique et attendre un clic de l'utilisateur pour effectuer un nouveau tour complet.

3 Gestion du clavier

Exercice 3.1 (Lettres et formes) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher des lignes séparant l'écran en quatre zones de même taille.
2. Afficher au centre de la première un rectangle si l'on a tapé R, au centre de la deuxième zone un carré si l'on a tapé C, au centre de la troisième zone un disque si l'on a tapé D et au centre de la dernière zone un cercle pour n'importe quelle autre saisie au clavier. On utilisera la fonction `wait_space_letter()`.

Exercice 3.2 (Clavier et couleur) Ecrivez une fonction réalisant les constructions suivantes :

1. Afficher une ligne horizontale blanche et une ligne verticale blanche permettant de séparer l'écran en quatre zones de même taille.
2. Mettre la zone en haut à gauche en rouge.
3. Déplacer la couleur de zone en zone à l'aide des flèches directionnelles.
4. Le programme se termine au bout de 10 changements.

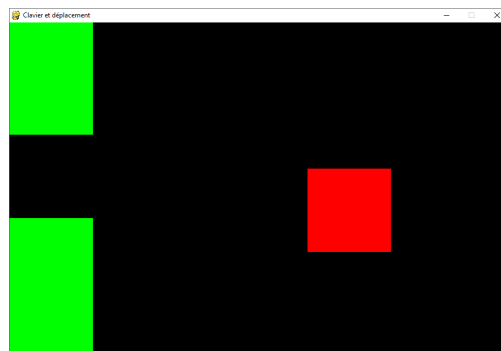
On écrira en particulier les fonctions

- * `affiche_zones()` pour afficher les quatre zones,
- * `affiche_couleur_zone(i, j)` pour afficher la couleur d'une zone,
- * `efface_couleur_zone(i, j)` pour effacer la couleur d'une zone,
- * `deplacement(i, j)` pour effectuer le déplacement de zone en zone.

Le couple (i, j) représente les coordonnées d'une zone. Par exemple, $(0, 0)$ pour la zone en haut à gauche, $(1, 0)$ pour la zone en haut à droite, $(0, 1)$ pour la zone en bas à gauche et $(1, 1)$ pour la zone en bas à droite.

Exercice 3.3 (Clavier et déplacement)

1. Afficher une bande verticale verte de largeur 150 à gauche de l'écran en laissant un trou de hauteur 150 (voir ci-dessous).
2. Afficher un carré rouge de côté 150 en dehors de cette bande.
3. Déplacer le carré sur la partie noire à l'aide des flèches directionnelles afin de le placer dans le trou précédent.
4. Le programme se termine une fois le carré bien positionné.
5. Rendre le placement du trou et la position initiale du carré aléatoire.



4 Un projet pas à pas : réalisation d'un jeu

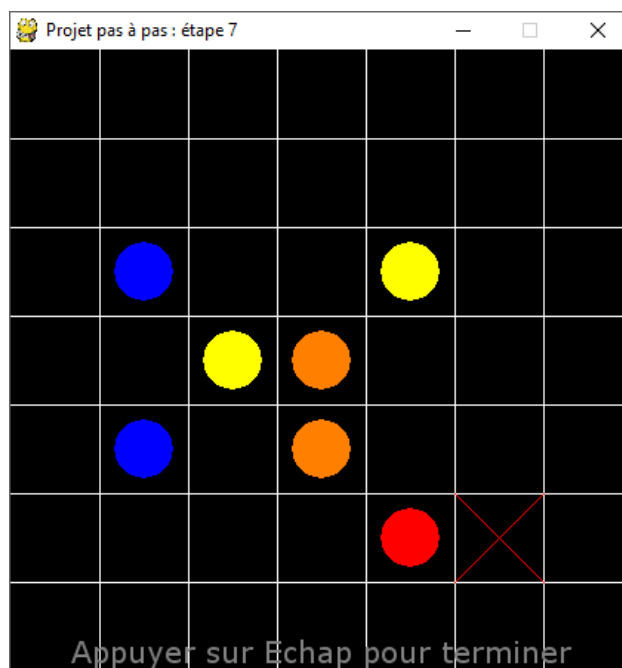
But du jeu : trouver, sur un quadrillage donné et en un nombre de coups fixé, une case qui a été choisie aléatoirement par l'ordinateur.

Pour aider le joueur, un code couleur a été défini :

- * rouge lorsque l'on a cliqué sur une case située à une distance 1 de la case voulue,
- * orange pour une case située à une distance 2,
- * jaune pour une case située à une distance 3 et
- * bleu pour une case située à une distance supérieure à 4.

Une croix indique la victoire lorsque l'on a cliqué sur la bonne case.

Une présentation finale de ce jeu est donnée par la copie d'écran ci-dessous :



Ce projet, détaillé en plusieurs étapes progressives, permet de mettre en place l'organisation nécessaire à la menée de projets plus ambitieux.

Etape 1 : Programmer une fonction permettant de construire un quadrillage 5×5 dans une fenêtre graphique de taille 500×500 .

Etape 2 : Programmer une fonction qui attend un clic et affiche un disque de couleur blanche et de rayon 30 centré sur la case cliquée.

Etape 3 : Modifier les deux fonctions précédentes afin de pouvoir les appeler dans une fonction principale `main()` sachant qu'au bout de 5 clics le programme se termine.

Etape 4 : Ajouter les fonctionnalités suivantes :

- * l'ordinateur choisit au hasard une case du quadrillage (ce qui revient, bien sûr, à choisir au hasard un point P correspondant au centre d'une case),
- * la partie s'arrête lorsque l'on a joué cinq coups ou lorsque l'on a cliqué sur la case choisie par l'ordinateur.

On pourra commencer par traiter le cas où la partie s'arrête uniquement si l'on a cliqué sur la case choisie par l'ordinateur.

Etape 5 : Modifier le programme afin qu'il affiche un disque de couleur blanche et de rayon 30 si la case cliquée n'est pas celle choisie par l'ordinateur et une croix rouge sinon.

Etape 6 : Ajouter une fonction renvoyant la distance entre la case cliquée et celle choisie par l'ordinateur. Intégrer au programme cette fonction afin d'afficher un disque rouge (resp. orange, jaune ou bleu) quand la distance vaut 1 (resp. 2, 3 ou 4 et plus).

Etape 7 : Modifier le programme afin que l'on puisse facilement modifier la taille des cases, la taille du quadrillage le rayon des disques et le nombre de coups à jouer.